

Strong Lens Inference, Explained From Zero

A complete beginner’s guide to the project — the physics, the statistics,
the code, and how to read every plot

Sayed Atique Newaz

Noureen Alam Meem

Mashuk Khan

July 2026

Abstract

This document explains the entire project assuming *no* prior knowledge: not of astronomy, not of Bayesian statistics, not of neural networks. It walks through what gravitational lensing is, what problem we are solving, how the code solves it (with the important pieces of source code shown and explained line by line), and then — plot by plot — what each figure in the project means, how to read it, and what ours actually shows. It accompanies the formal project report (`report/report.pdf`); read this one first.

Contents

1	The big picture: what are we even doing?	2
2	The physics: gravitational lensing in plain words	2
2.1	Mass bends light	2
2.2	What we see depends on alignment	2
2.3	The eight (or nine) numbers we infer	3
3	The statistics: Bayesian inference from zero	3
3.1	Uncertainty as probability	3
3.2	Why the textbook route fails here	3
3.3	The SBI trick: learn the inverse from examples	3
4	The code, part 1: simulating fake universes	4
4.1	Drawing random galaxies: the prior	4
4.2	From parameters to picture	4
4.3	Making it look real: noise	4
4.4	The dataset	5
5	The code, part 2: the neural networks	6
5.1	The summary network: 4096 pixels $\rightarrow$ 48 numbers	6
5.2	The inference network: 48 numbers $\rightarrow$ a full distribution	7
5.3	Training: what the loss means	7
6	The plots, part 1: does it work? (accuracy)	8
6.1	Parameter recovery: truth vs. estimate	8
6.2	The closed loop: posterior predictive check	9
7	The plots, part 2: can we trust the error bars? (calibration)	9
7.1	Simulation-based calibration (SBC)	9
7.2	Contraction and z-scores: how much did we learn, and is it biased?	11
7.3	One complete inference, visualized	12

8	The plots, part 3: the physics exam (κ and the mass-sheet degeneracy)	12
8.1	A question with a known answer: “I don’t know”	12
8.2	The verdict	13
9	Everything at a glance	15
10	Glossary	15

1 The big picture: what are we even doing?

Imagine you find a photograph of a strange circle of light in the sky. You are told: *a massive galaxy bent the light of another galaxy behind it, and this ring is the result*. The photograph is blurry and noisy. Your task:

From this one picture, work out the physical properties of the galaxy that bent the light — how heavy it is, how squashed its shape is, where exactly the background galaxy sits behind it — and say how sure you are about each number.

That is the whole project. Three things make it hard:

1. **The physics only runs forward.** Given the galaxy’s properties, we can compute the picture (that is a simulation). But there is no formula that takes the picture and returns the properties.
2. **The picture is noisy.** A real detector adds random noise, so even the forward direction is not exact — many slightly different pictures could come from the same galaxy.
3. **We need error bars, not just numbers.** “The mass is 1.2” is much less useful than “the mass is 1.20 ± 0.01 ”. Science runs on uncertainties.

Key idea

We solve it by **simulation-based inference (SBI)**: simulate tens of thousands of fake galaxies and their pictures, then train a neural network to go *backwards* — from picture to a full probability distribution over the galaxy’s properties. Train once, then answer for any new picture in milliseconds.

2 The physics: gravitational lensing in plain words

2.1 Mass bends light

Einstein’s general relativity says mass curves space, and light follows that curvature. A galaxy is heavy enough ($\sim 10^{11}$ times the mass of the Sun) that light passing near it gets visibly deflected — the galaxy acts like a huge, slightly imperfect glass lens floating in space.

2.2 What we see depends on alignment

Put a distant “source” galaxy almost exactly behind a nearby “lens” galaxy:

- **Perfect alignment** → the source’s light is smeared into a complete circle: an *Einstein ring*.
- **Slight offset** → the ring breaks into stretched *arcs*, or the source appears *multiple times* in one image.

Crucially, the distortion is not random — it encodes the lens:

What you measure in the image	What it tells you
Radius of the ring/arcs	The mass of the lens (Einstein radius θ_E)
How the arcs are squashed	The lens galaxy’s shape (e_1, e_2)
An extra overall stretch	Gravity of <i>neighbouring</i> structures (γ_1, γ_2)
Where the arcs sit	Where the source really is (x_s, y_s)
How thick the arcs are	How big the source is (R_s)

2.3 The eight (or nine) numbers we infer

Bundling those together, our unknown parameter vector is

$$\boldsymbol{\theta} = \left(\underbrace{\theta_E}_{\text{mass}}, \underbrace{e_1, e_2}_{\text{lens shape}}, \underbrace{\gamma_1, \gamma_2}_{\text{external tug}}, \underbrace{x_s, y_s}_{\text{source position}}, \underbrace{R_s}_{\text{source size}} \right) [+ \kappa].$$

The optional ninth number κ (“external convergence”) is a thin uniform sheet of extra mass in front of everything. It matters for a special reason explained in Section 8: physics *proves* it cannot be measured from a single image, so it lets us test whether our method is honest about what it cannot know.

3 The statistics: Bayesian inference from zero

3.1 Uncertainty as probability

Bayesian inference is bookkeeping for beliefs. Before seeing data you have a **prior** $p(\boldsymbol{\theta})$: which parameter values are plausible at all. The simulator defines the **likelihood** $p(\mathbf{x} | \boldsymbol{\theta})$: how probable is the observed image $\mathbf{x}$ if the truth were $\boldsymbol{\theta}$. Bayes’ rule combines them into the **posterior** — what you believe *after* seeing the image:

$$\underbrace{p(\boldsymbol{\theta} | \mathbf{x})}_{\text{posterior}} = \frac{\overbrace{p(\mathbf{x} | \boldsymbol{\theta})}^{\text{likelihood}} \overbrace{p(\boldsymbol{\theta})}^{\text{prior}}}{p(\mathbf{x})}.$$

The posterior is the complete answer to our task: for every parameter it gives a best value *and* an uncertainty *and* all correlations between parameters (“if the mass is a bit higher, the source must be a bit smaller”).

3.2 Why the textbook route fails here

To use Bayes’ rule directly you must be able to *evaluate* the likelihood as a formula. Our image is produced by ray-tracing light through a lens model, blurring with the telescope optics, and adding two kinds of noise. There is no closed-form expression for $p(\mathbf{x} | \boldsymbol{\theta})$ you could type into a computer. Classical tools (like MCMC) that need that formula are stuck — or need expensive approximations, hours per lens.

3.3 The SBI trick: learn the inverse from examples

We *can* sample from the likelihood — that is literally what the simulator does. So:

1. Draw a random $\boldsymbol{\theta}^{(i)}$ from the prior.
2. Simulate its noisy image $\mathbf{x}^{(i)}$.
3. Repeat 50,000 times $\rightarrow$ a big table of $(\boldsymbol{\theta}, \mathbf{x})$ pairs.
4. Train a neural network so that, shown $\mathbf{x}$, it outputs a distribution close to the true $p(\boldsymbol{\theta} | \mathbf{x})$.

Key idea

The network never sees a formula for the likelihood — only examples. All the physics enters through the simulator. This is why the method is called *simulation-based* (or *likelihood-free*) inference, and because one training pays for all future images, it is *amortized*.

4 The code, part 1: simulating fake universes

Everything configurable lives in one file, `src/config.py`. The pipeline is run from the command line:

```
python main.py generate # simulate the dataset (CPU, ~1 minute)
python main.py train # train the network (GPU, ~2 hours)
python main.py evaluate # produce all the plots in figures/
```

4.1 Drawing random galaxies: the prior

The prior ranges are plain numbers in `config.py` — every one chosen so the simulated system is a physically valid, visibly lensed galaxy:

```
PRIOR = {
    "theta_E": (0.7, 1.6), # Einstein radius: keeps the ring inside the image
    "q": (0.6, 1.0), # lens axis ratio -> converted to (e1, e2)
    "phi": (0.0, 3.14), # lens orientation angle
    "gamma_ext": (0.0, 0.08), # external shear strength (it is always weak)
    "phi_ext": (0.0, 3.14), # external shear direction
    "x_s": (-0.3, 0.3), # source position
    "y_s": (-0.3, 0.3),
    "R_s": (0.05, 0.25), # source size (arc thickness)
    "kappa": (0.0, 0.20), # mass sheet -- only in the 9-parameter version
}
```

Notice we do *not* draw e_1, e_2 directly. We draw a squashing factor q and an angle ϕ — quantities with clear physical meaning — and convert (`src/simulator/parameters.py`):

```
def _ellipticity_from_q_phi(q, phi):
    """Axis ratio q + position angle phi -> (e1, e2).
    Guarantees every draw is a valid ellipse (e1^2 + e2^2 < 1)."""
    e = (1.0 - q) / (1.0 + q)
    return e * np.cos(2.0 * phi), e * np.sin(2.0 * phi)
```

This tiny function has a visible consequence: the ring-shaped clouds you will see in the prior plot (Figure 2) are exactly the geometry this conversion creates. They are correct by construction, not a bug.

4.2 From parameters to picture

The simulator (`src/simulator/lens_generator.py`) hands the parameters to `lenstronomy`, a standard astrophysics package, which solves the lens equation — “which point of the source does each pixel actually show?” — then blurs the result with the telescope’s point-spread function (PSF) and samples it onto a 64×64 pixel grid, exactly like a small telescope camera would.

4.3 Making it look real: noise

A real detector adds two kinds of randomness, and we add both (`src/simulator/noise.py`):

```
def add_noise(clean_image, rng=None):
    """Gaussian background + Poisson shot noise, exactly like a real CCD."""
    background_noise = image_util.add_background(clean_image,
                                                sigma_bkd=C.BACKGROUND_RMS)
    poisson_noise = image_util.add_poisson(clean_image,
                                           exp_time=C.EXP_TIME)
    return clean_image + background_noise + poisson_noise
```

Background noise is the constant hiss of the sky and the electronics. *Poisson (shot) noise* exists because light arrives as individual photons: brighter pixels receive more photons and therefore fluctuate more in absolute terms. Getting the noise level right mattered enormously: our first datasets were too faint (peak signal-to-noise ≈ 4) and the network could not learn the subtle parameters at all. After a brightness sweep we reached a realistic median peak SNR of ≈ 16 .

4.4 The dataset

50,000 training + 2,000 validation + 300 test systems are simulated in parallel on all CPU cores and stored to disk, so training never touches the simulator again. Figure 1 shows what the network actually gets as input.

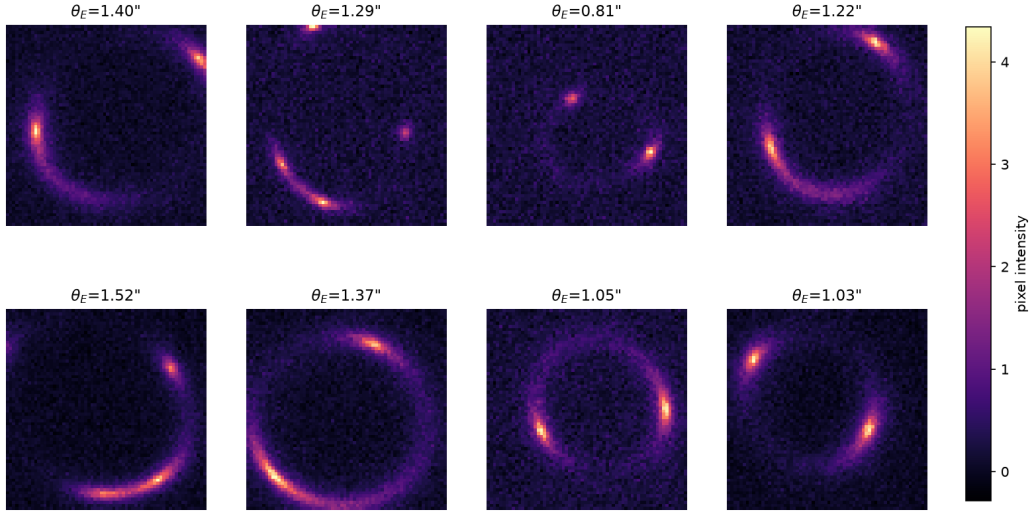


Figure 1: **Eight training images.** Each panel is one simulated “telescope photo” (with noise) labelled by its true Einstein radius θ_E . Note the variety: complete rings, broken arcs, thick and thin, centred and offset. This variety is the point — the network must learn to read *any* of these.

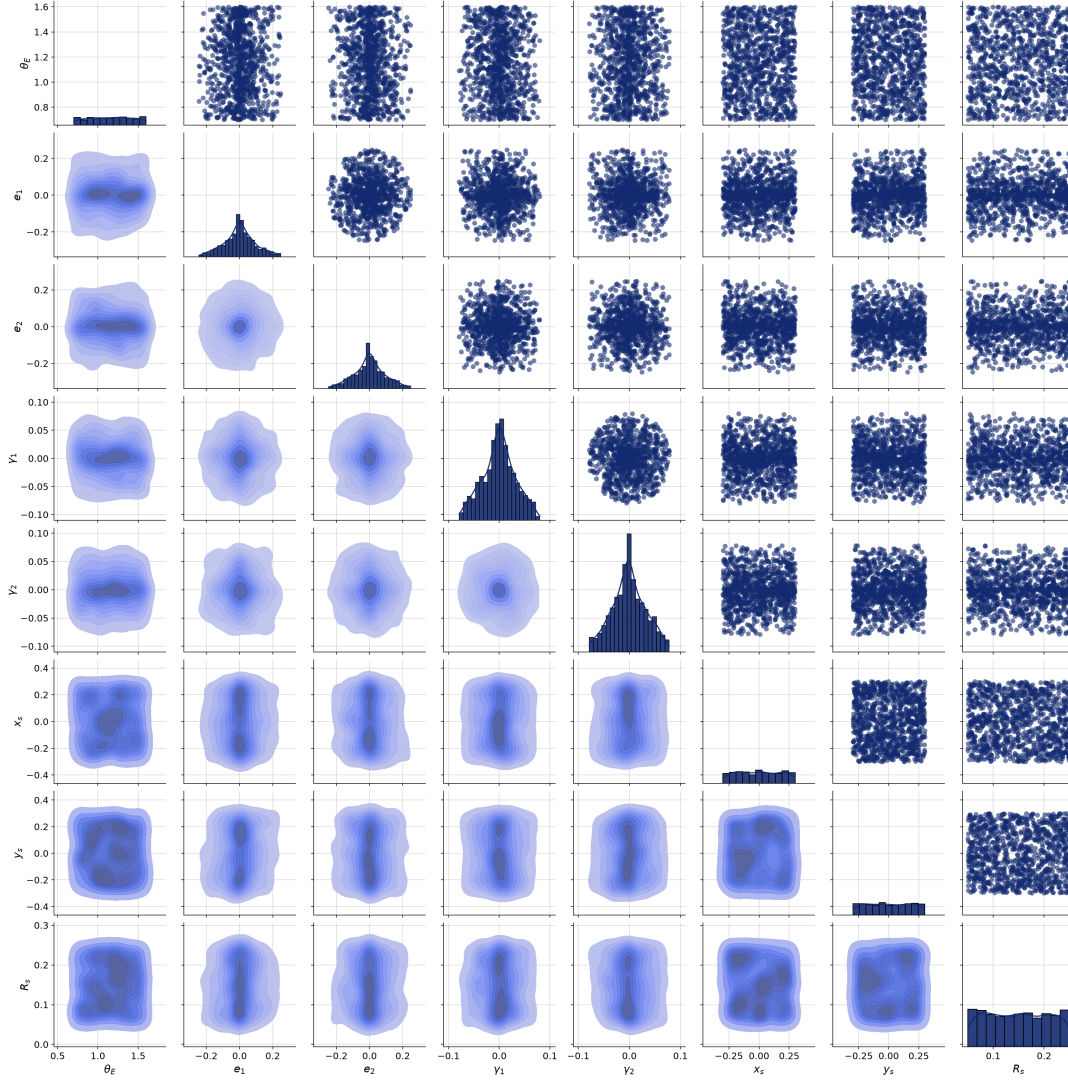


Figure 2: **The prior, visualized.** Diagonal panels: the distribution of each parameter across the training set (flat, as designed). Off-diagonal panels: every pair of parameters plotted against each other. The ring/disc shapes in the (e_1, e_2) and (γ_1, γ_2) panels come from drawing (strength, angle) pairs and converting to Cartesian components — the code shown above.

5 The code, part 2: the neural networks

The posterior estimator is two networks trained together end to end.

5.1 The summary network: 4096 pixels $\rightarrow$ 48 numbers

A 64×64 image is 4096 numbers, most of them noise. The summary network is a small **convolutional neural network (CNN)** — the standard architecture for images — that learns to compress each image into 48 numbers that retain the information about θ (`src/models/bayesflow_model.py`):

```
self.net = keras.Sequential([
    layers.Rescaling(1.0 / C.IMAGE_SCALE), # bring pixels near 0(1)

    layers.Conv2D(32, 3, padding="same", activation="relu"),
    layers.Conv2D(32, 3, padding="same", activation="relu"),
    layers.MaxPooling2D(), # 64 -> 32 pixels
    layers.BatchNormalization(),
```

```

layers.Conv2D(64, 3, padding="same", activation="relu"),
layers.Conv2D(64, 3, padding="same", activation="relu"),
layers.MaxPooling2D(), # 32 -> 16
layers.BatchNormalization(),

layers.Conv2D(128, 3, padding="same", activation="relu"),
layers.Conv2D(128, 3, padding="same", activation="relu"),
layers.MaxPooling2D(), # 16 -> 8
layers.BatchNormalization(),

layers.GlobalAveragePooling2D(), # -> 128 numbers
layers.Dense(128, activation="relu"),
layers.Dense(summary_dim), # -> 48 numbers
])

```

How to read this if you have never seen a CNN: each `Conv2D` layer slides many small 3×3 “feature detectors” across the image; early layers learn to respond to simple things (edges, bright spots), later layers to combinations of them (arc segments, ring curvature). Each `MaxPooling2D` halves the resolution, so by the last block the network is looking at the picture as a whole. Nobody tells it *which* features matter — it discovers, during training, whatever 48 numbers best predict the parameters.

5.2 The inference network: 48 numbers $\rightarrow$ a full distribution

The second network is a **conditional normalizing flow** (a *coupling flow* of depth 4). The intuition: it is a flexible, invertible, learnable “warping machine”. It takes simple random noise (a standard Gaussian) and warps it — in a way that depends on the 48-number summary — into the shape of the posterior. Because the warp is invertible, the network can both *evaluate* how probable a given θ is and *generate* new posterior samples cheaply. Sampling 2,000 plausible parameter sets for one image takes milliseconds.

This is what a plain “CNN that predicts 8 numbers” could never do: the flow outputs the entire joint distribution — widths, asymmetries, and correlations between parameters — not just a best guess.

5.3 Training: what the loss means

Training (`src/models/train.py`) shows the networks batches of $(\theta, \mathbf{x})$ pairs and adjusts the weights to maximize the probability the flow assigns to the *true* θ of each image — equivalently, to minimize the *negative log-posterior density*. If that loss goes down, the network’s distributions are becoming both better centred and better sized.

```

early_stop = keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=10, restore_best_weights=True)

history = workflow.fit_offline(
    train_data, epochs=C.EPOCHS, batch_size=C.BATCH_SIZE,
    validation_data=val_data, callbacks=[early_stop])

```

Two safety mechanisms are visible here. The *validation set* (2,000 images the optimizer never trains on) tells us whether the network is genuinely learning or just memorizing. *Early stopping* watches that validation loss and halts training if it stops improving — a guard against memorization (“overfitting”).

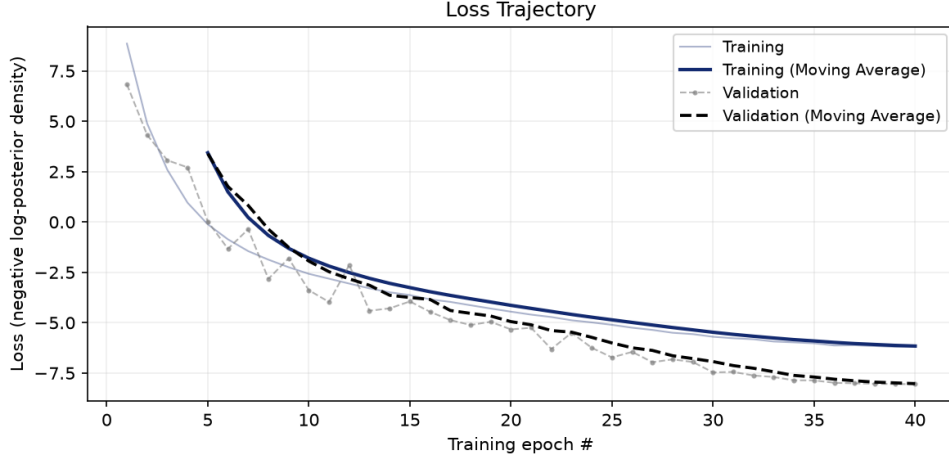


Figure 3: **The loss trajectory.** Thin lines: per-epoch loss; thick lines: moving average. *How to read it:* both curves should fall together. If the (dashed) validation curve turned upward while the training curve kept falling, the network would be memorizing — overfitting. *What ours shows:* both fall smoothly for all 40 epochs and validation ends *below* training at -8.05 — no overfitting at all; early stopping never had to intervene. (Our early 8,000-image pilot *did* overfit; scaling the dataset cured it.)

6 The plots, part 1: does it work? (accuracy)

All checks below use the 300 *test* systems — simulated lenses the network has never seen — with 2,000 posterior samples drawn per system.

6.1 Parameter recovery: truth vs. estimate

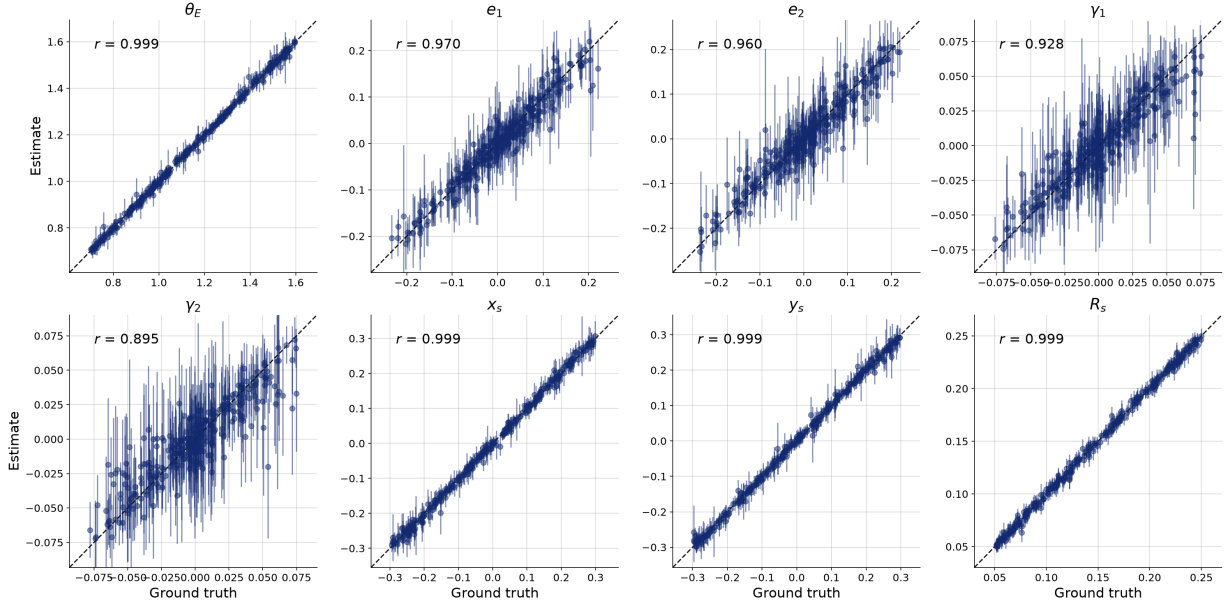


Figure 4: **Parameter recovery.** One panel per parameter. Horizontal axis: the true value used to simulate the image. Vertical axis: the network’s estimate (the posterior median), with the posterior spread (MAD) as an error bar. The dashed diagonal is perfection; r is the correlation between truth and estimate.

How to read it. Three things to look at, per panel:

- *Do the points follow the diagonal?* If yes, the estimates are accurate.
- *Is the cloud flat instead?* Then the image does not constrain that parameter — the network falls back towards the prior average (this is honest behaviour, not failure).
- *Are the error bars the right size?* They should be large exactly where the scatter is large.

What ours shows. A clean three-tier hierarchy that mirrors the physics:

Tier	Parameters	r	Why
Near-perfect	θ_E , x_s , y_s , R_s	0.999	they set ring size/position/thickness
Very good	e_1 , e_2	0.97 / 0.96	arc squashing is measurable at SNR 16
Good	γ_1 , γ_2	0.93 / 0.90	shear is tiny (≤ 0.08) and subtle

The shear error bars are visibly the widest — the network *knows* shear is its weakest measurement, which is exactly what we want from a posterior.

6.2 The closed loop: posterior predictive check

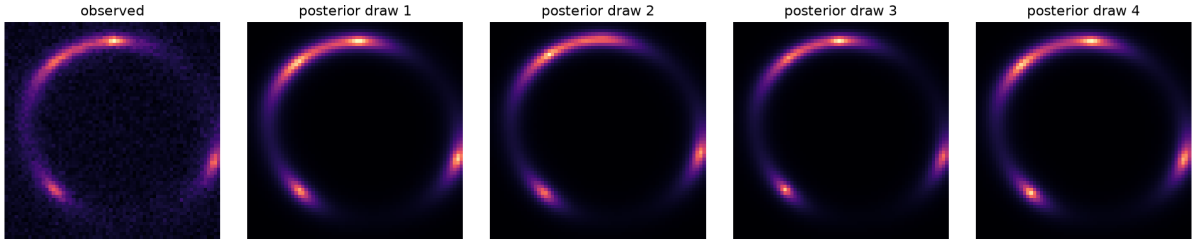


Figure 5: **Posterior predictive check.** Leftmost: one observed (noisy) test image. The other four panels: we drew four independent parameter sets from the network’s posterior for that image and re-simulated each one (noise-free).

How to read it. If the posterior is centred on the right parameters, pushing its samples back through the simulator must reproduce the observation. Differences *between* the four re-simulations visualize the remaining uncertainty. **What ours shows:** ring radius, the bright knots, and the light distribution are all reproduced; the draws differ only in fine knot brightness — precisely the detail that the noise genuinely erases. The network reconstructs what is knowable and stays uncertain about what is not.

7 The plots, part 2: can we trust the error bars? (calibration)

Accuracy is not enough. A method that says “ $\theta_E = 1.20 \pm 0.001$ ” when the truth is regularly 1.25 is *worse* than useless — it is confidently wrong. So we test the error bars themselves.

7.1 Simulation-based calibration (SBC)

The logic, in dice terms: if a weather service’s “70% chance of rain” is honest, it must rain on 70% of such days. Analogously, the true parameter must fall in the central 50% of the posterior 50% of the time, in the central 90% of it 90% of the time, and so on. Since we *simulated* the test systems, we know every truth and can check this directly: for each system, compute the *rank* of the truth within the 2,000 posterior samples. Honest posteriors $\Rightarrow$ ranks uniformly distributed between 0 and 1.

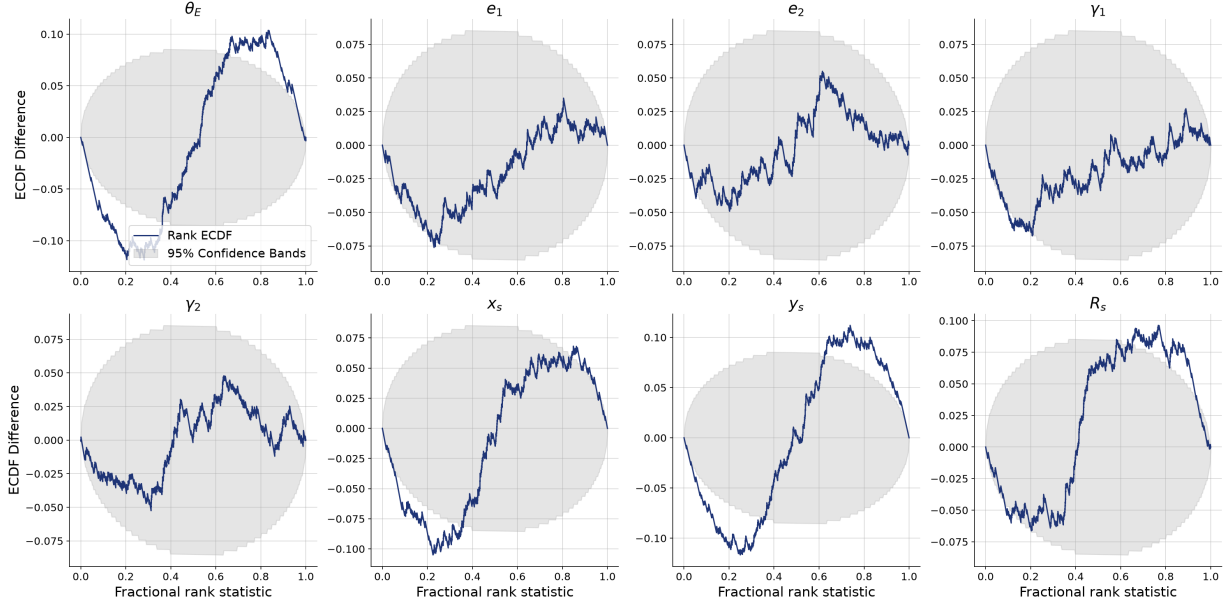


Figure 6: **SBC rank-ECDF difference.** For each parameter, the blue curve is the difference between the observed distribution of ranks and perfect uniformity; the grey ellipse is the band inside which an honest method stays 95% of the time.

How to read it. The shape of the excursion is a diagnosis:

- *Inside the band* $\rightarrow$ calibrated. Trust the error bars.
- *Dip-then-peak S-shape* (too few extreme ranks, too many central ones) $\rightarrow$ posteriors too *wide* $\rightarrow$ **underconfident**. Conservative; the safe failure.
- *The mirror image* (rank pile-up at the extremes) $\rightarrow$ posteriors too *narrow* $\rightarrow$ **overconfident**. The dangerous failure.

What ours shows. Ellipticity and shear: inside the band — calibrated. The four best-measured parameters (θ_E, x_s, y_s, R_s): a mild S-shape outside the band — their error bars are slightly too wide. Two important qualifications: (1) this is the harmless direction — our intervals never *overclaim*; (2) the shape is *identical* at 20,000 and 50,000 training images, so more data does not fix it — it is a limitation of the network’s capacity (the 48-number summary / depth-4 flow), and we know exactly which knob to turn next. Finding, quantifying, and diagnosing this is what calibration tests are *for*.

7.2 Contraction and z-scores: how much did we learn, and is it biased?

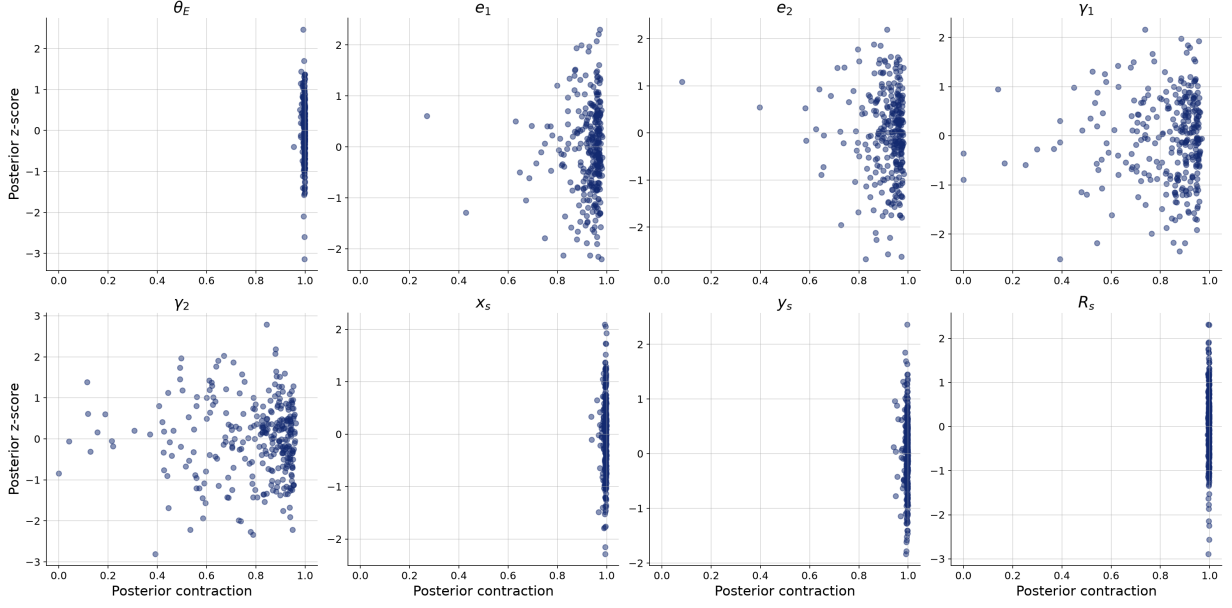


Figure 7: **Posterior z-score vs. posterior contraction**, one dot per test system. Horizontal axis: contraction = $1 - \sigma_{\text{posterior}}^2 / \sigma_{\text{prior}}^2$ (0 = the image taught us nothing; 1 = it removed all prior uncertainty). Vertical axis: $(\text{posterior mean} - \text{truth}) / \sigma_{\text{posterior}}$, i.e. the error measured in units of the claimed uncertainty.

How to read it. The ideal is the upper-right “funnel”: contraction near 1 and z-scores spread tightly around 0 (roughly within ± 2 , like a standard normal). Low contraction is acceptable *if* z-scores remain centred — that just means “the image contains little information about this parameter, and the network says so honestly”. **What ours shows:** the morphology-dominant parameters sit at contraction ≈ 1 with centred z-scores — maximal, unbiased information gain. Ellipticity and shear reach intermediate contraction with z-scores still centred: less information, honestly reported, no bias.

7.3 One complete inference, visualized

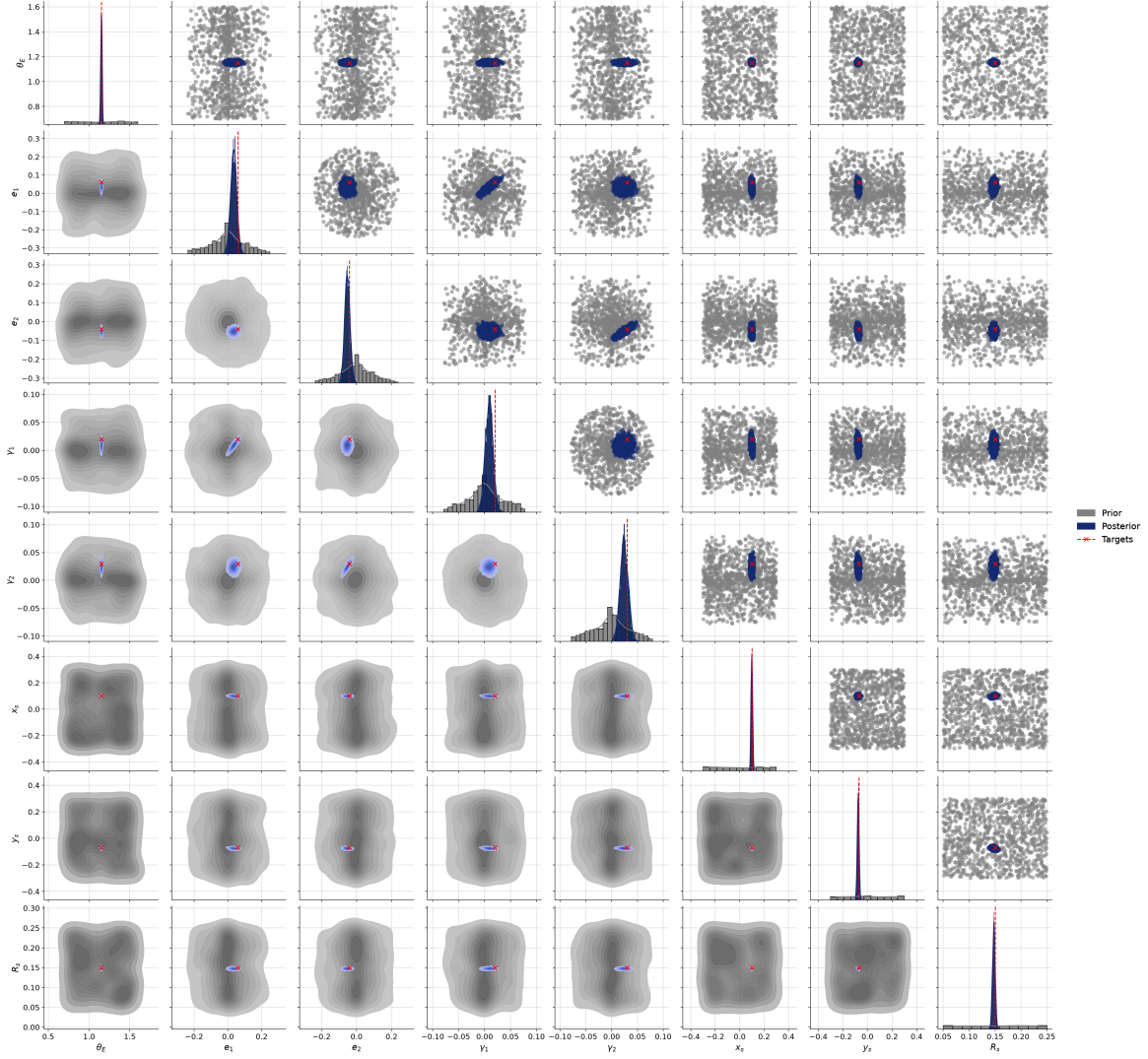


Figure 8: **The corner plot for one “observed” system.** Grey: the prior (what we believed before looking). Blue: 2,000 posterior samples (after looking at one image). Red: the true values used to simulate that image. Diagonal: per-parameter distributions; off-diagonal: every pair of parameters jointly.

How to read it. Three checks: the blue blob should be much smaller than the grey cloud (we learned a lot), it should contain the red marker (we learned the *right* thing), and its tilts reveal parameter correlations (“these two trade off against each other”). **What ours shows:** dramatic contraction onto the truth — e.g. $\theta_E = 1.152 \pm 0.008$ against a truth of 1.150 — with the widest surviving spread in the shear components, and clearly tilted ellipses showing the network reports *joint* structure, not just eight separate error bars.

8 The plots, part 3: the physics exam (κ and the mass-sheet degeneracy)

8.1 A question with a known answer: “I don’t know”

Lensing theory contains a celebrated blind spot. Add a perfectly uniform sheet of mass κ in front of the lens and simultaneously rescale the (unobservable) source: the resulting image is *almost*

exactly the same. Consequence: no method, however clever, can measure κ from a single image. This is the **mass-sheet degeneracy**.

That makes κ a perfect exam question. We add it as a ninth parameter, regenerate 50,000 simulations, retrain from scratch, and ask: does the network *pretend* to measure κ (bad — overconfident nonsense), or does it *report the impossibility* (good — honest posterior)?

8.2 The verdict

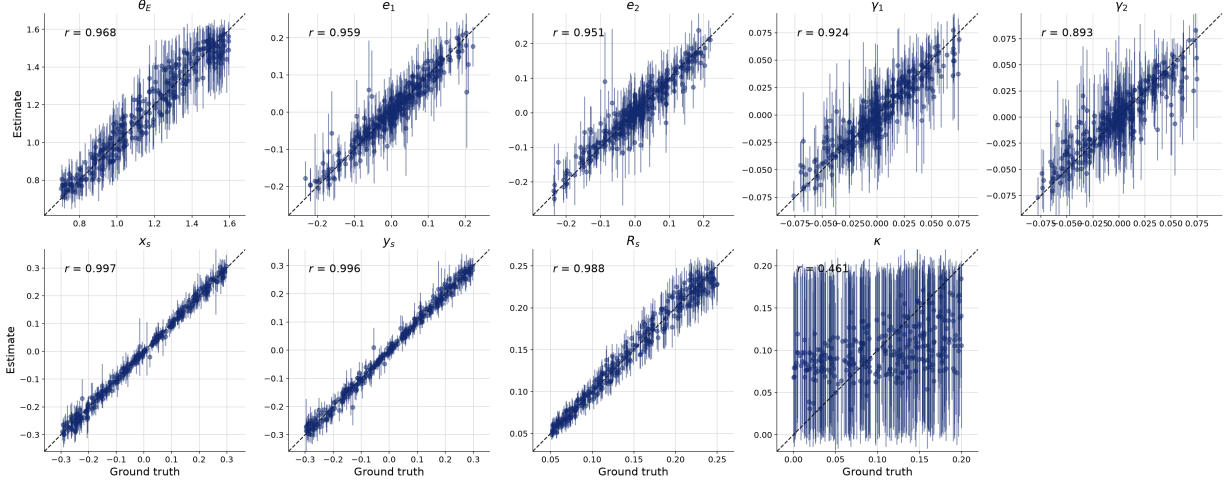


Figure 9: **Recovery for the 9-parameter model.** Compare with Figure 4. The new κ panel (bottom right) is a flat cloud: estimates hover near the prior average whatever the truth is, with error bars spanning nearly the whole prior — the network says “I don’t know”. Meanwhile the θ_E panel has visibly widened (R^2 : 1.00 $\rightarrow$ 0.94); every other panel is unchanged.

Three quantitative signatures, all exactly as theory demands:

1. **κ stays unknown.** Prior spread 0.058 $\rightarrow$ average posterior spread 0.048: a contraction of only 16%. One image teaches the network almost nothing about κ — correctly.
2. **The degeneracy direction is discovered.** Within each individual posterior, κ and θ_E are correlated at -0.96 : the network learned the precise trade-off (more mass sheet $\Leftrightarrow$ smaller intrinsic Einstein radius) that the lens equations predict. Nobody told it. And this correlation *sharpened* as our data improved (-0.83 in the noisy pilot $\rightarrow -0.96$ now) — a sharper posterior hugs the ridge more tightly.
3. **The damage is surgical.** Only θ_E pays for the ambiguity (R^2 1.00 $\rightarrow$ 0.94); all seven other parameters are untouched. The degeneracy couples κ to the mass — and to nothing else.

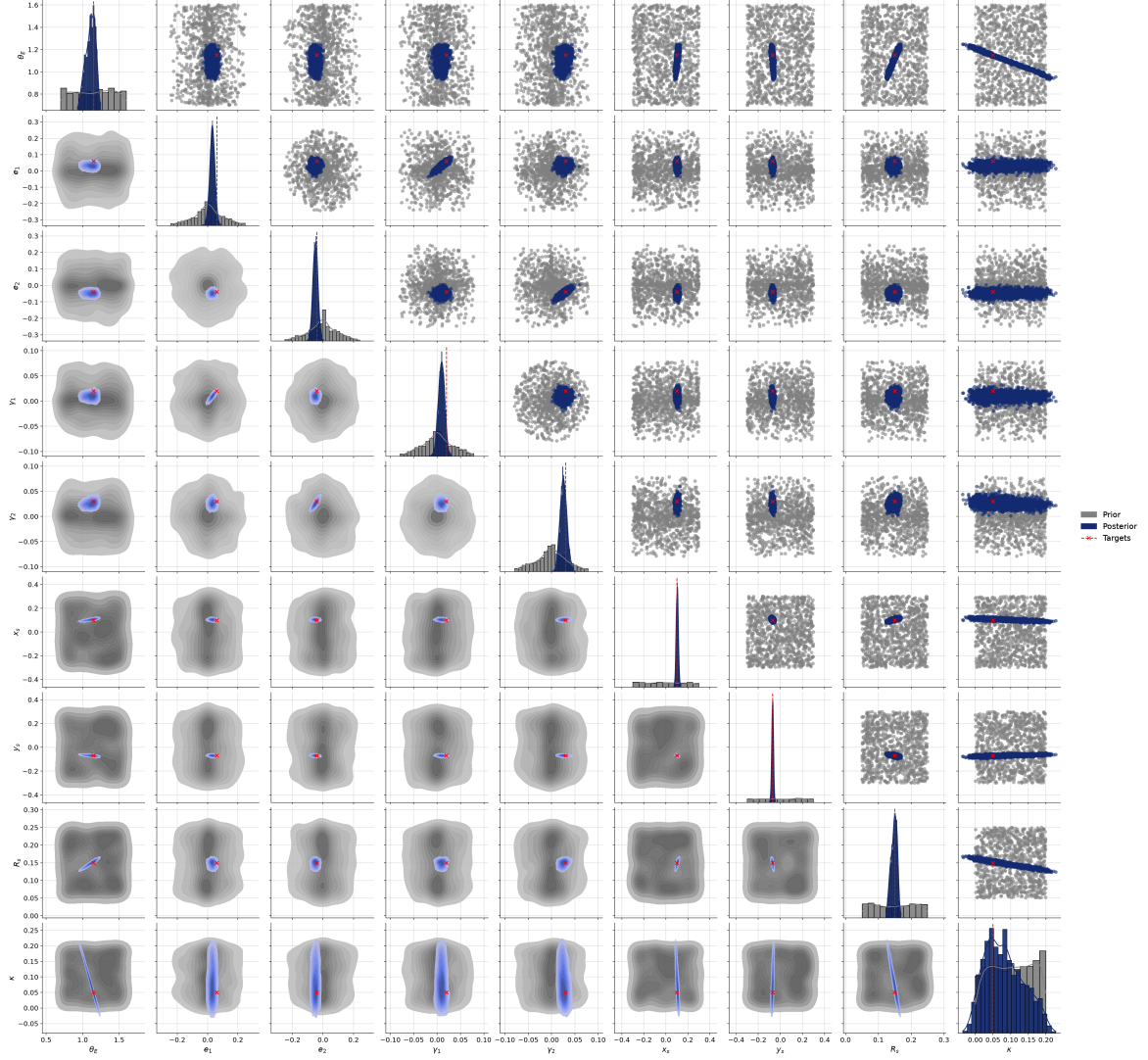


Figure 10: **The corner plot with κ , for the same fiducial system.** Look at two places. The κ diagonal panel: the blue posterior is nearly as wide as the grey prior — honest ignorance. The (κ, θ_E) panel: the posterior is a thin *tilted* ridge — the degeneracy line itself, drawn by the network. Compare θ_E 's width here (± 0.065) with the 8-parameter fit of the same image (± 0.008): the unmeasurable mass sheet inflates the mass uncertainty by a factor ~ 8 , exactly as it must.

Key idea

The network *rediscovered a theorem of gravitational lensing* purely from simulated examples. For us this is the strongest evidence in the whole project that the learned posteriors are physically meaningful — they don't just fit numbers, they encode the structure of the theory.

9 Everything at a glance

Question	Plot that answers it	Our answer
Can it recover the parameters?	recovery	$r = 0.999$ for mass/position/-size; 0.96–0.97 shape; 0.90–0.93 shear
Are the error bars honest?	SBC rank-ECDF	Yes for shape/shear; slightly too cautious (never overconfident) for the rest
How much does one image teach?	contraction/z-score	Nearly everything about morphology; honestly partial about shear
Does the posterior regenerate the data?	posterior predictive	Yes, down to the bright knots
Does it respect known physics?	κ diagnostics	Mass-sheet degeneracy reproduced: κ unknown, $\text{corr}(\kappa, \theta_E) = -0.96$
How fast is it?	—	Simulate 52k images: ~ 1 min; train: ~ 2 h once; infer: milliseconds per lens

10 Glossary

Prior What you consider plausible before seeing data.

Likelihood How probable the observed data is for given parameters — here defined implicitly by the simulator.

Posterior Updated beliefs after seeing the data; the full answer, including uncertainties and correlations.

Amortized inference Train a network once on simulations; afterwards any new observation is analysed in milliseconds.

Summary network The CNN that compresses each 64×64 image into 48 informative numbers.

Normalizing flow / coupling flow An invertible network that warps simple Gaussian noise into the posterior distribution — and can therefore both sample from it and score it.

Overfitting Memorizing the training set instead of learning the pattern; detected by a rising validation loss.

SBC (simulation-based calibration) The rank-uniformity test of whether credible intervals have their claimed coverage.

Underconfident / overconfident Error bars too wide (safe) / too narrow (dangerous).

Posterior contraction How much narrower the posterior is than the prior — the information gained from the data.

Einstein radius θ_E The angular radius of the ring; the lensing measure of the lens mass.

External shear γ The gentle extra stretch from the gravity of neighbouring structures.

External convergence κ / mass-sheet degeneracy A uniform mass sheet that provably cannot be measured from one image; our built-in honesty test.

Companion documents: the formal write-up is `report/report.pdf`; per-run numbers are in `RESULTS.md`; the short markdown version of this guide is `EXPLANATION.md`.